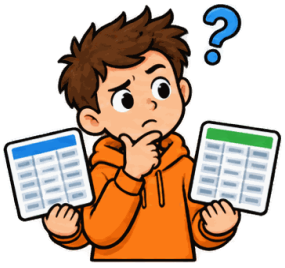
# Ticketkontrolle beim Schulfest

Lineare Suche — Schritt für Schritt durch eine Liste

**AUSGANGSSITUATION**



Die SMV hat **300 Tickets** für das Schulfest verkauft. Jedes Ticket trägt eine Nummer. Mia steht am Einlass mit der Liste der verkauften Nummern – in der Reihenfolge, in der sie verkauft wurden, also unsortiert. Ein Gast zeigt Ticket **2290**. Mia geht die Liste von oben nach unten durch. Der nächste Gast zeigt Ticket **3333**. Diese Nummer wurde nie verkauft – das Ticket ist gefälscht.

**Wie viele Nummern muss Mia vergleichen – und bei welchem Gast dauert die Kontrolle länger: beim echten Ticket oder beim gefälschten?**

**Was tippst du?** Kreuze an, bevor du zählst:

☐ beim echten Ticket ☐ beim gefälschten Ticket ☐ gleich lang

Mias Liste – die ersten zwölf verkauften Tickets (gesetzte Annahme)

| Index | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Nummer | 4711 | 1204 | 3390 | 2875 | 4102 | 1560 | 3921 | 2290 | 4488 | 1733 | 3057 | 2601 |

## 1 Aufgabe 1 — Aufwärmen

**Nenne** aus dem Gedächtnis den Python-Ausdruck, mit dem du aus der Liste `tickets` die Nummer **3390** erhältst.

**SO GEHT'S**

### So sucht ein Rechner: die lineare Suche

1. Beginne beim **ersten Element** (Index 0).
2. Vergleiche das Element mit dem gesuchten Wert.
3. Gleich? Dann ist der Wert gefunden – merke dir den Index und **höre auf**.
4. Nicht gleich? Gehe zum nächsten Element und vergleiche wieder.
5. Ist die Liste zu Ende und nichts hat gepasst, gilt: **nicht vorhanden**.

**Beispiel:** Suche nach 3390 in Mias Liste. Vergleich 1: 4711 – nein. Vergleich 2: 1204 – nein. Vergleich 3: 3390 – ja, gefunden bei Index 2. Die Suche brauchte **3 Vergleiche**.

## 2 Aufgabe 2

**Vervollständige** das Ablaufprotokoll für die Suche nach Ticket **2290**. Die ersten zwei Schritte sind eingetragen. Trage darunter die Anzahl der Vergleiche und den Index ein.

| Vergleich Nr. | Index | Nummer in der Liste | Ist es 2290? (ja / nein) |
|---|---|---|---|
| 1 | 0 | 4711 | nein |
| 2 | 1 | 1204 | nein |
| 3 | | | |
| 4 | | | |
| 5 | | | |
| 6 | | | |
| 7 | | | |
| 8 | | | |
| 9 | | | |
| 10 | | | |

Anzahl der Vergleiche: _____ 2290 steht bei Index: _____

## 3 Aufgabe 3

**Erkläre**, warum Mia bei Ticket **3333** erst nach der letzten Nummer der Liste sicher sein kann, dass das Ticket gefälscht ist.

**Satzanfänge:** „Solange noch eine Nummer übrig ist, …“ · „Erst wenn …, dann …“

MERKE

### Bester und schlimmster Fall

Wie viele Vergleiche die lineare Suche braucht, hängt davon ab, **wo der Wert steht**. Steht er ganz vorn, reicht ein Vergleich. Steht er ganz hinten oder fehlt er, muss jeder Eintrag angeschaut werden. Die Anzahl der Einträge nennt man **n**.

**bester Fall: 1 Vergleich · schlimmster Fall: n Vergleiche**

## 4 Aufgabe 4

**Bestimme** die Anzahl der Vergleiche für Mias Ausschnitt, für die ganze Liste und für den Fall, dass die SMV nächstes Jahr doppelt so viele Tickets verkauft.

| Fall | Beispiel | n = 12 | n = 300 | n = 1200 |
|---|---|---|---|---|
| bester Fall | Ticket 4711 | | | |
| schlimmster Fall | Ticket 3333 (gefälscht) | | | |

## 5  Aufgabe 5 — Partnerarbeit

**Dokumentiere** eine eigene Suche: Schreibe sechs vierstellige Ticketnummern in beliebiger Reihenfolge auf und wähle einen gesuchten Wert – er darf in der Liste stehen oder fehlen. Führe die lineare Suche durch und trage das Ergebnis ein. Dein Partner prüft dein Protokoll mit der Schrittfolge aus dem Infokasten.

| Index 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| | | | | | |

Gesucht: _____ Vergleiche: _____ Index (oder „fehlt“): _____

☐ vom Partner geprüft

**INFO**

### Bausteine lesen und zusammensetzen

Ein Python-Programm besteht aus Zeilen mit **Einrückung**: Eine Zeile, die mit einem Doppelpunkt endet, öffnet einen Block – alles, was zu ihr gehört, steht eine Stufe tiefer (4 Leerzeichen). `for i in range(len(liste))` lässt `i` alle Indizes von 0 bis zum letzten durchlaufen.

`return` **beendet die Funktion sofort** und gibt einen Wert zurück. Für „nicht gefunden“ nimmt man eine Zahl, die nie ein Index sein kann: `-1`.

## 6 Aufgabe 6 — Code bauen

**Implementiere** die Funktion `lineare_suche(liste, gesucht)` aus den 5 Bausteinen: Bringe sie in die richtige Reihenfolge und trage im Bauplan je Zeile den Buchstaben ein – in der Spalte seiner Einrückungsstufe. Jeder Baustein wird genau einmal gebraucht.

**A** `if liste[i] == gesucht:`
Treffer?

**B** `return -1`
Liste zu Ende: nicht gefunden

**C** `def lineare_suche(liste, gesucht):`
Funktion mit zwei Eingaben: die Liste und der gesuchte Wert

**D** `return i`
gefunden: Index zurückgeben, die Suche endet sofort

**E** `for i in range(len(liste)):`
i läuft über alle Indizes: 0, 1, 2, ...

Bauplan – Einrückungsstufe (eine Stufe = 4 Leerzeichen)

| Zeile | Stufe 0 | Stufe 1 | Stufe 2 | Stufe 3 | Stufe 4 |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | | | | | |
| 3 | | | | | |
| 4 | | | | | |
| 5 | | | | | |

**Wenn du nicht weiterkommst:** Womit beginnt jede Funktion? Welche Zeilen enden mit einem Doppelpunkt – und was muss darunter eine Stufe tiefer stehen?

## 7 Aufgabe 7 — Schreibtischtest

**Teste** dein Programm aus Aufgabe 6 auf dem Papier mit `liste = [7, 3, 9]` und `gesucht = 9`: Trage für jeden Schleifendurchlauf ein, was das Programm tut, und den Rückgabewert.

| i | liste[i] | liste[i] == gesucht? | Was passiert? |
|---|---|---|---|
| 0 | | | |
| 1 | | | |
| 2 | | | |

Rückgabewert: _____ Und mit `gesucht = 4`? Rückgabewert: _____ nach _____ Vergleichen



Zähle nicht die Zeilen der Tabelle, sondern die **Vergleiche**: Auch die letzte Nummer, die nicht passt, hat Mia angeschaut – das ist ein Vergleich.

## 8 Aufgabe 8 — Mias Frage

**Begründe** mit deinen Zahlen aus Aufgabe 2 und 4, bei welchem Gast Mias Kontrolle länger dauert – und was das für den Einlass mit 300 Tickets bedeutet. Vergleiche am Ende mit deinem Tipp vom Anfang.

**Satzanfänge:** „Beim echten Ticket braucht Mia …, beim gefälschten …“ · „Das liegt daran, dass …“ · „Für 300 Tickets heißt das …“

## 9 Aufgabe 9 — Vertiefung

**Erweitere** die Funktion aus Aufgabe 6 so, dass sie mitzählt, wie viele Vergleiche sie gemacht hat, und beides zurückgibt: `return index, vergleiche`. Schreibe die vollständige Funktion auf.

## 10 Aufgabe 10 — Zusatz

**Entwickle** ein Vorgehen, mit dem die SMV nach dem Fest herausfindet, welche verkauften Tickets **nicht eingelöst** wurden. Mia hat dafür eine zweite, ebenfalls unsortierte Liste mit allen am Einlass gescannten Nummern. Halte die Schritte in Stichpunkten fest und gib an, wie viele Vergleiche dein Vorgehen bei 300 verkauften und 300 gescannten Tickets höchstens braucht.

### PRÜFE DICH SELBST

- ☐ Steht in deinem Protokoll aus Aufgabe 2 genau einmal „ja“ – in der letzten ausgefüllten Zeile?
- ☐ Ist dein Wert für n = 1200 viermal so groß wie der für n = 300?
- ☐ Gibt deine Funktion für ein Ticket, das fehlt, die Zahl −1 zurück – und nicht `None`?